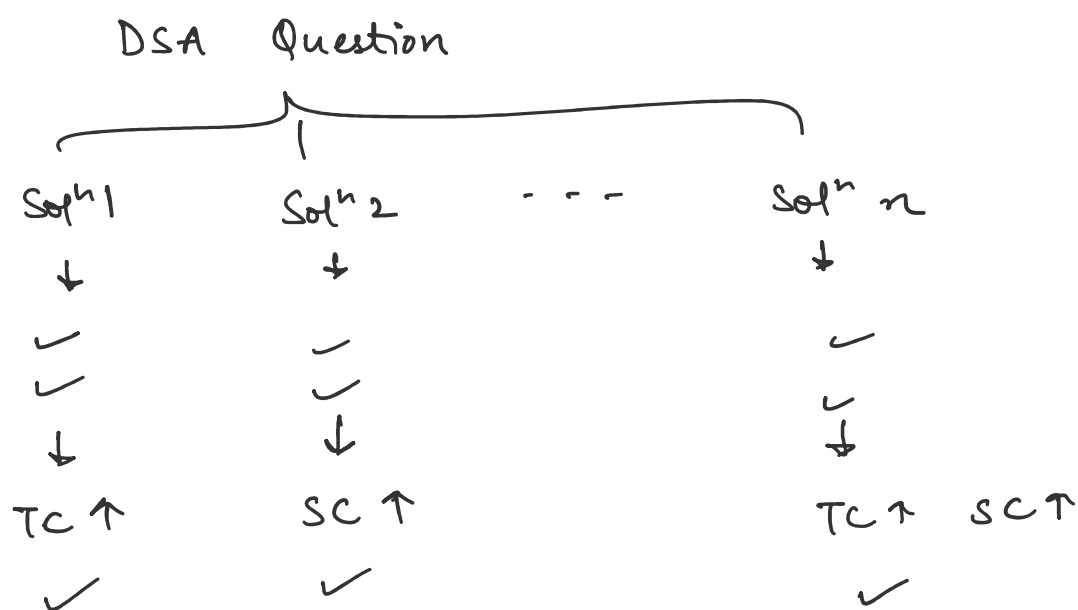


Strategy design Pattern ==



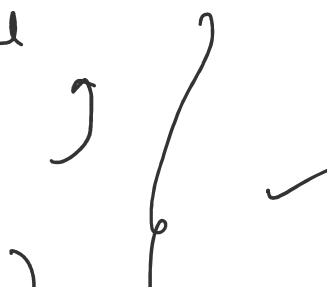
login Page

↓

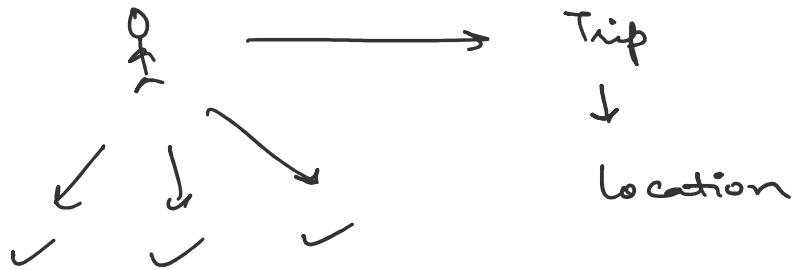
various opⁿs to login -

→ login via email

→ login via OTP



... login via ...
→ login via google



```
class Login
{
    ==
```

if or switch

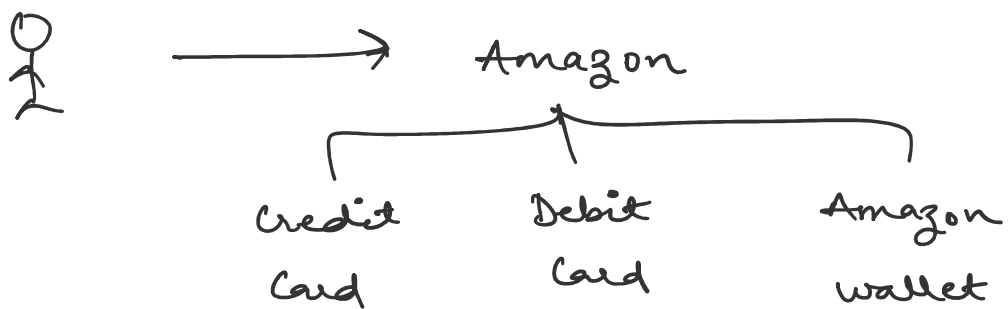
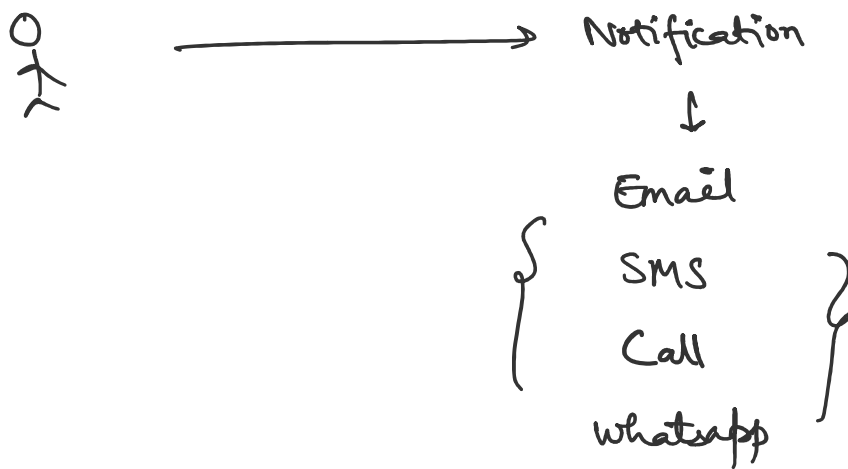
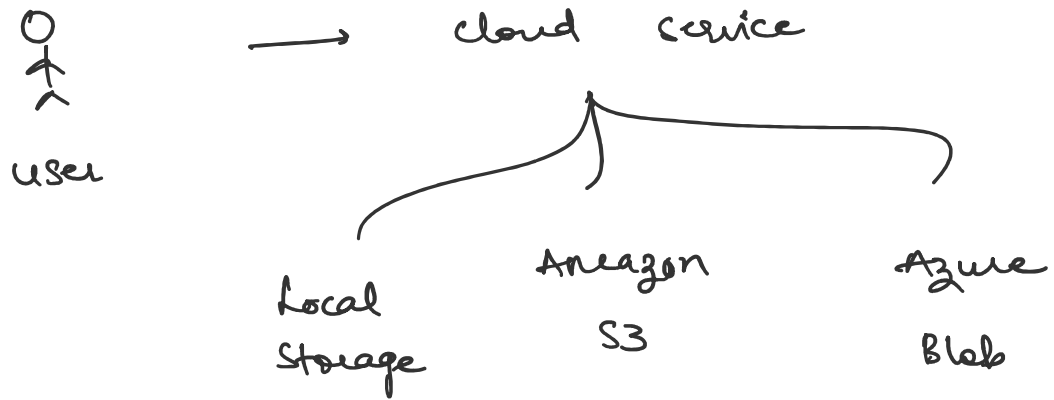
```
if ( type == "Email" )
    ↪ email()
```

```
else if ( type == "SMS" )
    ↪ sms()
```

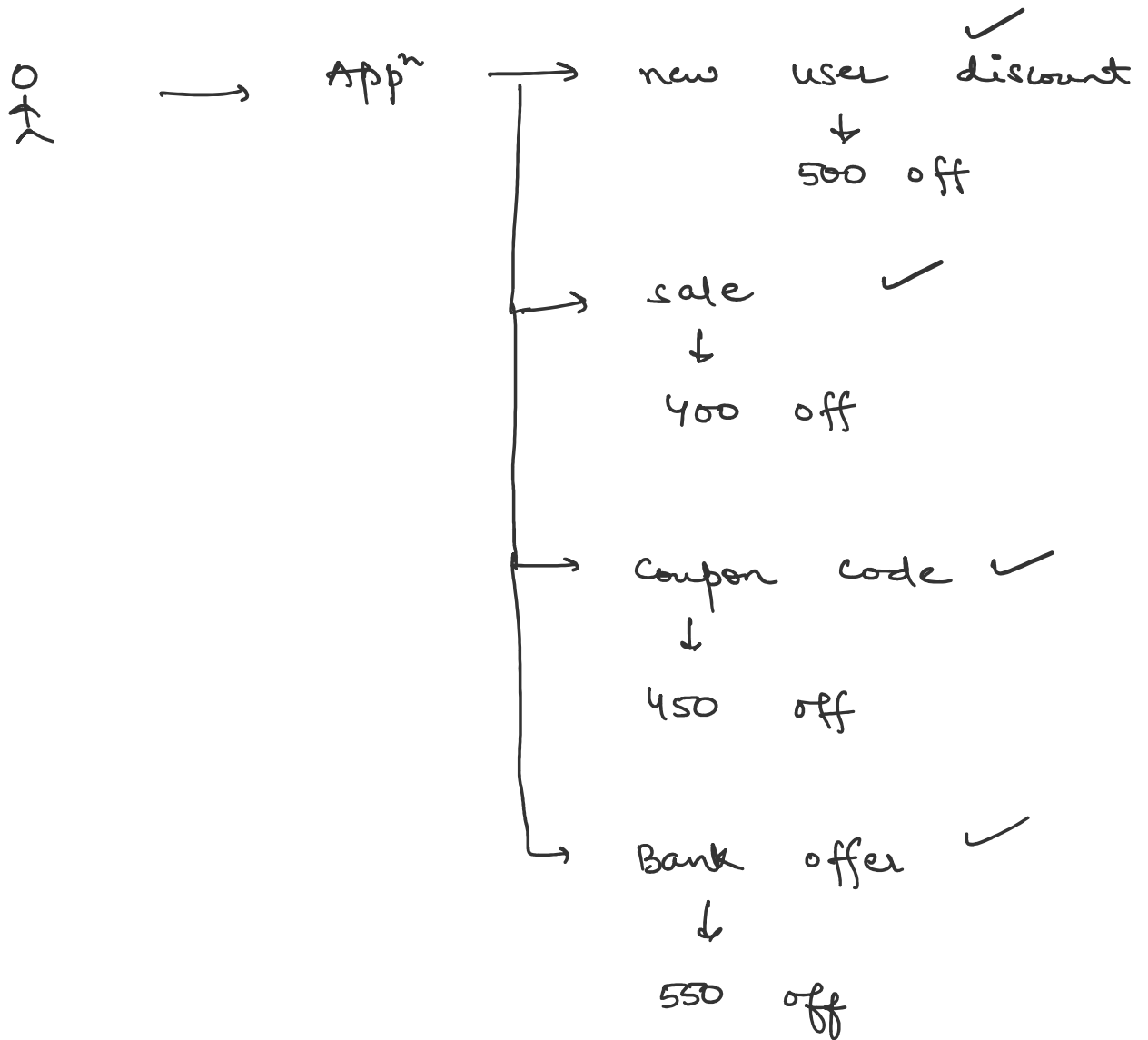
else

```
    google()
```

```
}
```



Discount Engine



Stick figure → Placing order

✓ Apply code

o CC } → ✓ ↑
o DC }

✓ interface DiscountStrategy
↳ applyDiscount()

Concrete Strategies → Class

↳ New user
↳ sale day discount
↳ Bank offer
↳ Coupon Discount

Context

↳ checkout service

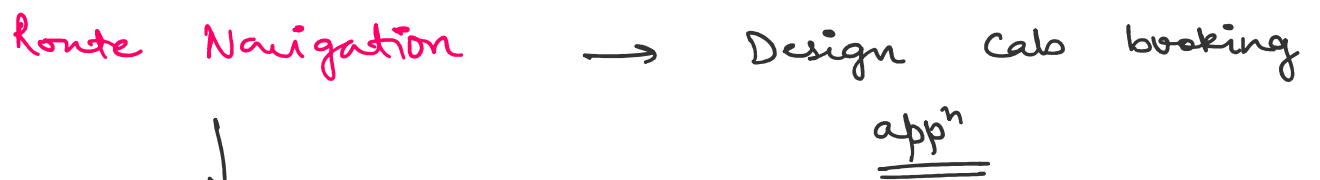
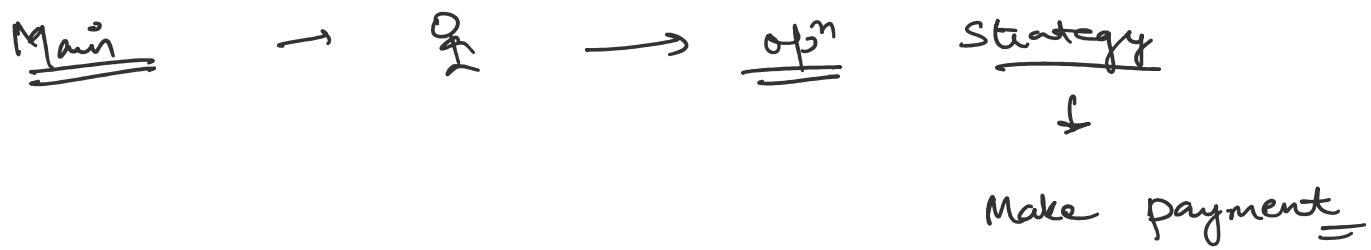
↓

→ constructor

→ set strategy

→ checkout

←



Google maps

- fastest route ✓
- Shortest route ✓
- Toll free route

✓ source → current location | Enter
 ✓ location

Destination $\rightarrow$ reach

Strategy interface



interface RouteStrategy

↳ calculateRoute (source, dest)

Concrete strategy -

→ Toll free

→ Shortest path

→ Fastest path

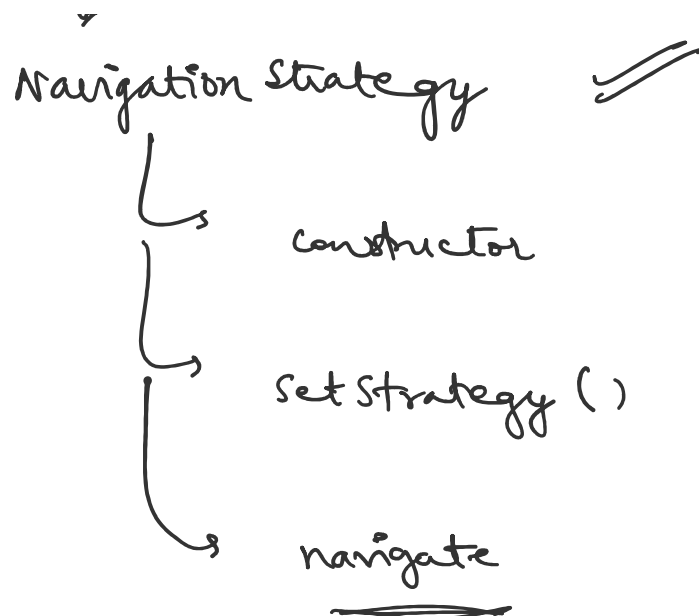


Context



Navigation Strategy





Main → client

Notes:

Strategy Design Pattern

Imagine you are in an interview.

- The interviewer gives you a problem (context).
- You, as the interviewee, can solve it using **different strategies** (brute force, dynamic programming, greedy, backtracking, etc.).

- The interviewer (client) doesn't care how you solve it, as long as you pick one working strategy.

In real software projects, we often encounter situations where the **same business operation can be performed in multiple ways**.

For example:

- A payment can be made using **UPI, Credit Card, Debit Card, or Net Banking**.
- A notification can be sent through **Email, SMS, or WhatsApp**.
- A file can be stored on **Local Disk, Amazon S3, or Azure Blob Storage**.
- A user can log in using **Password, Google, or Microsoft**.

Initially, developers usually implement these requirements using **if-else** or **switch** statements.

For example:

- If the payment method is **UPI**, execute UPI logic.
- Else if it is **Credit Card**, execute credit card logic.
- Else if it is **Debit Card**, execute debit card logic.
- Else if it is **Net Banking**, execute net banking logic.

This works when there are only a few options.

However, as the application grows, new requirements keep coming:

- Add **Wallet** payment.
- Add **Apple Pay**.
- Add **PayPal**.
- Add **Buy Now Pay Later**.

Every new option requires **modifying the same class again and again**.

As a result:

- The class becomes **large and difficult to understand**.
- Business logic for all implementations gets **mixed together**.
- Testing becomes **harder**.
- The risk of **breaking existing functionality** increases.
- The code **violates the Open/Closed Principle (OCP)** because existing code must be modified whenever a new implementation is added.

This is a **very common problem in enterprise applications**.

How Strategy Pattern Solves This Problem

Instead of putting every implementation inside **one class**, the **Strategy Pattern** separates **each implementation into its own class**.

The main class no longer contains all the business logic.

Instead, it simply **delegates the work to the selected strategy**. The **client chooses which strategy should be used**, and the **context executes it without knowing its internal implementation**.

As a result:

- Each implementation is **independent**.
- New strategies can be added **without changing existing code**.
- Existing strategies remain **untouched**.
- The application can **switch between strategies at runtime**.
- The code becomes **cleaner, easier to maintain, and easier to test**.

This follows the **Open/Closed Principle (OCP)** because **new behavior is added by creating new strategy classes instead of modifying existing ones**.

Real-World Example : Notification Service

An application needs to send notifications to users.

Different users prefer different communication channels:

- Email
- SMS
- Push Notification
- WhatsApp
- Slack

How Strategy Pattern Helps

Each notification channel becomes its own strategy. The notification service simply invokes the selected strategy to send the message. Adding support for a new notification channel does not affect the existing implementation.

Another Example : File Storage

An application allows users to upload files. Depending on deployment, files may be stored in:

- Local Disk
- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

How Strategy Pattern Helps

Each storage provider implements its own upload logic.

The upload service delegates the file storage operation to the selected storage strategy.

Changing the storage provider requires only selecting a different strategy.

Another Example : Authentication

Users can log in using different authentication methods:

- Username and Password
- Google Login
- Microsoft Login
- LDAP
- OAuth

How Strategy Pattern Helps

Each authentication mechanism becomes a separate strategy.

The authentication service invokes the chosen strategy without knowing how authentication is performed internally.

Another Example : Shipping Cost Calculation

Shipping charges vary depending on the selected delivery option:

- Standard Delivery

- Express Delivery
- Same-Day Delivery
- International Shipping

How Strategy Pattern Helps

Each pricing algorithm becomes its own strategy.

The shipping service calculates charges using the selected strategy.

Implementation of Strategy Pattern

The Strategy Pattern consists of **four components**.

1. Strategy Interface

Define a **common interface** that represents the operation which can have **multiple implementations**. Every strategy implements this interface.

2. Concrete Strategies

Create a **separate class** for each implementation. Each class contains **one specific algorithm or business rule**. Every strategy performs the **same task**, but in **its own way**.

3. Context

The **Context** uses a strategy to perform the task. It **does not know how the task is implemented**. Its only responsibility is to **delegate the work to the currently selected strategy**. The strategy can even be **changed while the application is running**.

4. Client

The **Client** chooses which strategy should be used. It creates or selects the appropriate strategy and passes it to the **Context**. The Context simply executes it.

Payment Processing

Imagine you're building an e-commerce application like Amazon or Flipkart.

When a customer proceeds to checkout, they can choose from multiple payment methods:

- Credit Card
- Debit Card
- UPI
- Wallet

Each payment method has a completely different implementation. Initially, the Checkout Service contains multiple if-else statements to determine which payment logic to execute.

As the business grows, new payment methods such as **PayPal**, **Apple Pay**, or **Buy Now Pay Later (BNPL)** need to be supported.

Every new payment method requires modifying the Checkout Service, making it larger, harder to maintain, and violating the **Open/Closed Principle (OCP)**.

Solution

Using the **Strategy Pattern**, each payment method becomes a separate strategy. The Checkout Service simply delegates the payment to the selected strategy without knowing how the payment is processed.

When a new payment method is introduced, we simply create a new strategy class. The Checkout Service remains unchanged.

Step 1: Create the Strategy Interface

All payment methods perform the same task: **pay the amount**.

```
public interface PaymentStrategy {  
    void pay(double amount);  
}
```

Step 2: Create Concrete Strategies

Credit Card Payment

```
public class CreditCardPayment implements PaymentStrategy {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using Credit  
Card.");  
    }  
}
```

Debit Card Payment

```
public class DebitCardPayment implements PaymentStrategy {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using Debit  
Card.");  
    }  
}
```

UPI Payment

```
public class UpiPayment implements PaymentStrategy {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using UPI.");  
    }  
}
```

Wallet Payment

```
public class WalletPayment implements PaymentStrategy {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid ₹" + amount + " using Wallet.");  
    }  
}
```

Step 3: Create the Context

The **CheckoutService** does not know how a payment is processed. It simply delegates the payment to the selected strategy.

```
public class CheckoutService {  
    private PaymentStrategy paymentStrategy;  
  
    public CheckoutService(PaymentStrategy paymentStrategy) {  
        this.paymentStrategy = paymentStrategy;  
    }  
  
    public void checkout(double amount) {  
        paymentStrategy.pay(amount);  
    }  
  
    public void setPaymentStrategy(PaymentStrategy  
paymentStrategy) {  
        this.paymentStrategy = paymentStrategy;  
    }  
}
```

Step 4: Client Chooses the Strategy

```
public class Main {  
    public static void main(String[] args) {  
        CheckoutService checkout = new CheckoutService(new  
UpiPayment());  
        checkout.checkout(1200);  
        checkout.setPaymentStrategy(new CreditCardPayment());  
        checkout.checkout(2500);  
        checkout.setPaymentStrategy(new WalletPayment());  
        checkout.checkout(800);  
    }  
}
```

/* *

Output :

Paid ₹1200.0 using UPI.

Paid ₹2500.0 using Credit Card.

Paid ₹800.0 using Wallet.

* */

Discount Engine

Imagine you're building an e-commerce application like Amazon or Flipkart. During checkout, customers often try different offers to get the best price before placing an order.

For example, a customer might compare:

- Sale Day Discount
- Bank Offer Discount
- Coupon Discount
- New User Discount

Each discount follows different business rules and calculates the final price differently.

Without the Strategy Pattern, the **Checkout Service** would contain multiple if-else or switch statements to determine which discount to apply.

As the application grows, new discounts such as **Student Discount**, **Festive Coupon**, or **Loyalty Discount** need to be added.

Every new discount requires modifying the Checkout Service, making it larger, harder to maintain, and violating the **Open/Closed Principle (OCP)**.

How Strategy Pattern Helps

Each discount calculation is implemented as a separate strategy. The **Checkout Service** simply delegates the discount calculation to the selected strategy without knowing how the discount is calculated.

During checkout, if the customer wants to compare different offers, the application simply changes the discount strategy at runtime. The **Checkout Service** remains the same.

When a new discount type is introduced, only a new strategy class is created. No changes are required in the Checkout Service.

Step 1: Create the Strategy Interface

Every discount strategy performs the same task: **calculate the final price after applying the discount**.

```
public interface DiscountStrategy {  
    double applyDiscount(double amount);  
}
```

Step 2: Create Concrete Strategies

New User Discount

Flat ₹500 off on the first order.

```
public class NewUserDiscount implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double amount) {  
        return amount - 500;  
    }  
}
```

Sale Day Discount

10% off during special sales like Big Billion Days or Prime Day.

```
public class SaleDayDiscount implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double amount) {  
        return amount * 0.90; // 10% discount  
    }  
}
```

Bank Offer Discount

15% instant discount on selected bank cards.

```
public class BankOfferDiscount implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double amount) {  
        return amount * 0.85; // 15% discount  
    }  
}
```

Coupon Discount

Flat ₹300 off using a coupon code.

```
public class CouponDiscount implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double amount) {  
        return amount - 300;  
    }  
}
```

Step 3: Create the Context

The Checkout Service doesn't know how discounts are calculated. It simply delegates the calculation to the selected strategy.

```
public class CheckoutService {  
    private DiscountStrategy discountStrategy;  
  
    public CheckoutService(DiscountStrategy discountStrategy) {  
        this.discountStrategy = discountStrategy;  
    }  
  
    public void checkout(double amount) {  
        double finalAmount =  
discountStrategy.applyDiscount(amount);  
  
        System.out.println("Original Amount : ₹" + amount);  
        System.out.println("Final Amount : ₹" + finalAmount);  
    }  
  
    public void setDiscountStrategy(DiscountStrategy  
discountStrategy) {
```

```
        this.discountStrategy = discountStrategy;
    }
}
```

Step 4: Client Chooses the Strategy

```
public class Main {
    public static void main(String[] args) {
        double orderAmount = 5000;
```

```
        CheckoutService checkout = new CheckoutService(new
        NewUserDiscount());
```

```
        System.out.println("Applying New User Discount");
        checkout.checkout(orderAmount);
```

```
        System.out.println();
```

```
        System.out.println("Customer removes it and checks Sale
        Day Discount");
        checkout.setDiscountStrategy(new SaleDayDiscount());
        checkout.checkout(orderAmount);
```

```
        System.out.println();
```

```
        System.out.println("Customer now selects Bank Offer");
        checkout.setDiscountStrategy(new BankOfferDiscount());
        checkout.checkout(orderAmount);
```

```
        System.out.println();
```

```
        System.out.println("Customer finally applies Coupon
        Code");
        checkout.setDiscountStrategy(new CouponDiscount());
```

```
        checkout.checkout(orderAmount);
    }
}
```

```
/* *
```

Output :

Applying New User Discount

Original Amount : ₹5000.0

Final Amount : ₹4500.0

Customer removes it and checks Sale Day Discount

Original Amount : ₹5000.0

Final Amount : ₹4500.0

Customer now selects Bank Offer

Original Amount : ₹5000.0

Final Amount : ₹4250.0

Customer finally applies Coupon Code

Original Amount : ₹5000.0

Final Amount : ₹4700.0

```
* */
```

Route Navigation

A maps application can calculate routes in different ways:

- Fastest Route (minimum time)
- Shortest Route (minimum distance)

- Toll-Free Route (avoid toll roads)

How Strategy Pattern Helps

Each routing algorithm becomes a separate strategy.

The navigation service simply executes the selected routing strategy.

Step 1: Create the Strategy Interface

Every routing strategy performs the same task: **calculate a route between two locations.**

```
public interface RouteStrategy {  
    void calculateRoute(String source, String destination);  
}
```

Step 2: Create Concrete Strategies

Fastest Route

Uses traffic data to find the quickest route.

```
public class FastestRouteStrategy implements RouteStrategy {  
    @Override  
    public void calculateRoute(String source, String destination) {  
        System.out.println("Calculating the fastest route from " +  
source + " to " + destination + ".");  
    }  
}
```

Shortest Route

Uses minimum distance to calculate the route.

```
public class ShortestRouteStrategy implements RouteStrategy {  
    @Override  
    public void calculateRoute(String source, String destination) {  
        System.out.println("Calculating the shortest route from " +  
source + " to " + destination + ".");  
    }  
}
```

Toll-Free Route

Avoids toll roads while calculating the route.

```
public class TollFreeRouteStrategy implements RouteStrategy {  
    @Override  
    public void calculateRoute(String source, String destination) {  
        System.out.println("Calculating the toll-free route from " +  
source + " to " + destination + ".");  
    }  
}
```

Step 3: Create the Context

The **NavigationService** doesn't know how the route is calculated. It simply delegates the route calculation to the selected strategy.

```
public class NavigationService {  
    private RouteStrategy routeStrategy;  
  
    public NavigationService(RouteStrategy routeStrategy) {  
        this.routeStrategy = routeStrategy;  
    }  
}
```

```

    }

    public void navigate(String source, String destination) {
        routeStrategy.calculateRoute(source, destination);
    }

    // Change strategy at runtime
    public void setRouteStrategy(RouteStrategy routeStrategy) {
        this.routeStrategy = routeStrategy;
    }
}

```

Step 4: Client Chooses the Strategy

This example demonstrates **changing the route preference at runtime**, just like Google Maps.

```

public class Main {
    public static void main(String[] args) {
        String source = "Bangalore";
        String destination = "Chennai";

        NavigationService navigation = new NavigationService(new
FastestRouteStrategy());

        System.out.println("User selects Fastest Route");
        navigation.navigate(source, destination);

        System.out.println();

        System.out.println("User changes preference to Toll-Free
Route");
        navigation.setRouteStrategy(new TollFreeRouteStrategy());
        navigation.navigate(source, destination);
    }
}

```

```

System.out.println();

System.out.println("User changes preference to Shortest
Route");
navigation.setRouteStrategy(new ShortestRouteStrategy());
navigation.navigate(source, destination);
    }
}

```

```

/* *

```

Output :

User selects Fastest Route

Calculating the fastest route from Bangalore to Chennai.

User changes preference to Toll-Free Route

Calculating the toll-free route from Bangalore to Chennai.

User changes preference to Shortest Route

Calculating the shortest route from Bangalore to Chennai.

```

* */

```

When to Use Strategy Pattern

Use the Strategy Pattern when:

- The **same task can be performed using multiple algorithms or business rules.**
- Different implementations need to be **selected at runtime.**

- You want to **eliminate long if-else or switch statements**.
- New implementations are expected to be **added frequently**.
- You want to make the application **easier to maintain and extend**.

Advantages

- Eliminates large **if-else** and **switch** statements.
- Makes code **easier to read and maintain**.
- Allows adding **new strategies without modifying existing code**.
- Supports **changing behavior at runtime**.
- Encourages **loose coupling** between the Context and the strategies.
- Makes individual strategies **easier to test**.

Disadvantages

- Introduces **more classes** because each strategy has its own implementation.
- The **client** is responsible for choosing the appropriate strategy.
- May be **unnecessary** if there is only **one implementation** and no possibility of future variation.